Lab 2: ROS2 control and Inverse Kinematics

Introduction to Robotics DT512A

Temur Mohammad Nasr, Nicolaus Hentschel

Task 2: State Controller

We worked on top of the provided StateController in my_simple_controllers & filled in the TODOs

On configure:

- For every moving joint in the chain we look up its index in joint_names_. The state interfaces are ordered like joint_names_, which is not necessarily the same order as the chain
- The FK solver is created here as a member, so update() doesn't allocate memory inside the control loop
- For TF we use a realtime_tools::RealtimePublisher on /tf instead of a normal TransformBroadcaster because a normal one can block inside update(U), the message is set up once here (base_link -> state_controller_fk)
- We added 3 parameters in parameters.yaml: base_frame, tip_frame and fk_frame

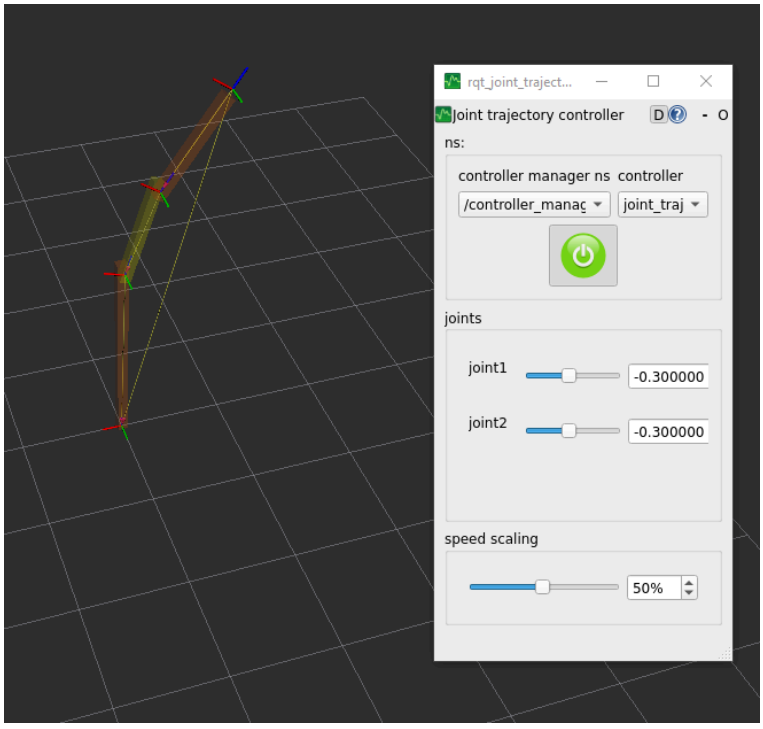
update():

- Throttle -> only computes when the publish period has passed
- Reads the joint positions from the position state interfaces
- JntToCart -> pose of the tool tip -> TF message -> published if the realtime publisher isn't busy

Verification: we moved the robot with QRT and compared our frame with tool_link from robot_state_publisher:

q1, q2	state_controller_fk	tool_link
0.24, 0.00	[0.452, 0.200, 3.796], 0.240 rad about y	[0.452, 0.200, 3.796], 0.240 rad about y
-0.30, -0.30	[-0.831, 0.200, 3.635], -0.600 rad about y	[-0.831, 0.200, 3.635], -0.600 rad about y

-> Identical in both poses, also when both joints are bent.



RViz and RQT with q1=-0.30, q = -0.30

```
At time 1793650598.784268726
- Translation: [-0.831, 0.200, 3.635]
- Rotation: in Quaternion (xyzw) [0.000, -0.296, 0.000, 0.955]
- Rotation: in RPY (radian) [0.000, -0.600, 0.000]
- Rotation: in RPY (degree) [0.000, -34.377, 0.000]
- Matrix:
  0.825 -0.000 -0.565 -0.831
  0.000  1.000 -0.000  0.200
  0.565  0.000  0.825  3.635
  0.000  0.000  0.000  1.000
```

```
At time 1793650479.554311614
- Translation: [-0.831, 0.200, 3.635]
- Rotation: in Quaternion (xyzw) [0.000, -0.296, 0.000, 0.955]
- Rotation: in RPY (radian) [0.000, -0.600, 0.000]
- Rotation: in RPY (degree) [0.000, -34.377, 0.000]
- Matrix:
  0.825 -0.000 -0.565 -0.831
  0.000  1.000 -0.000  0.200
  0.565  0.000  0.825  3.635
  0.000  0.000  0.000  1.000
```

Tf2_echo for state_controller_fk and tool_link in the pose above -> same values.

Task 3: Jacobians

New package lab02 with the node test_jacobians, everything is in main:

- Reads the URDF file -> KDL::Tree with 2 joints and 4 segments
- KDL::TreeJntToJacSolver works directly on the tree so no chain is needed
- JntToJac(q, jac, tipFrame) with the end effector frame as a parameter (tipFrame = tool_link)
- KDL::Jacobian already holds an Eigen matrix inside, so the conversion is just Eigen::MatrixXd J = jac.data
- The Jacobian is 6x2

KDL output at q = [0, 0]:

```
1.9   1
  0   0
  0   0
  0   0
  1   1
  0   0
```

Analytical Jacobian:

```
        joint1                          joint2
vx  [ L1·cos q1 + L2·cos(q1+q2)     L2·cos(q1+q2) ]
vy  [ 0                             0             ]
vz  [ -L1·sin q1 - L2·sin(q1+q2)    -L2·sin(q1+q2)]
ωx  [ 0                             0             ]
ωy  [ 1                             1             ]
ωz  [ 0                             0             ]
```

At q = [0, 0] -> vx = 1.9 and 1, ωy = 1 and 1, the rest 0 -> same as KDL. vy, ωx and ωz are always 0 since the arm only moves in the x-z plane.

Comparison for 4 test configurations, error = norm of (J_KDL - J_analytical):

Q1, q2	Error
0, 0	2.48e-16
πi/2, 0	1.53e-16
π/4, π/4	2.96e-16
0.3, -1.2	1.11e-16

-> only rounding errors

Questions:

- What does “changing the base frame” mean?
    - The Jacobian maps joint velocities of the velocity of the end effector. That velocity needs to be written in a frame. To change the base frame means to write the same velocity but in another frame -> rotate both the angular part and the linear part. Where R is the rotation between frames. So the motion is the same, only the coordinates changed.

- What reference does KDL use?
    - The Jacobian from KDL is expressed in the base frame (base_link). The reference point at the end effector (tool_link). So the top 3 rows are the velocity of the tool tip written in base link coordinates -> geometric Jacobian. That is why it matched our derivative of the tip position,

Task 4: Inverse Kinematic Solver

Second node in lab02: test_ik, it advertises the service with our own service type SolveIK.srv:

```
# Request
string frame_name
geometry_msgs/Pose target_pose
---
# Response
bool success
sensor_msgs/JointState solution
```

Constructor:

- Parameters
- URDF (same as task 3=
- Publisher for JointState on ik_joint_states

Service callback:

1. Creates the FK solver (TreeFkSolverPos_recursive) and the Jacobian solver (TreeJntToJacSolver)
2. Random initial guess, uniform within the joint limits of each joint
3. Goal pose Tg as an Eigen::Affine3d from the position + quaternion in the request
4. Loop up to maxIter
    a. FK-> current pose Tc, converted to Eigen::Affine3d (via a quaternion)
    b. Pos error = goal position - current position, rotation error = $Rg \cdot Rc^T$ as AngleAxisd
    c. Converged if the pos error < posTol and the angle < rotTol

   d. Twist v = [position error, angle·axis], with Δt = 1. It is written in the base frame, same as the KDL Jacobian
   e. Jacobian J(q_c)
   f. Update q_c = q_c + λ·J⁺·v, with J⁺ from completeOrthogonalDecomposition().pseudoInverse().
   g. Publishes q_c on ik_joint_states
5. Returns the final q_c as a JointState + success

Damping: with damping k > 0 we use damped least squares instead of the plain pseudo inverse. Near a singularity J loses rank and the plain pseudo inverse divides by values close to 0 which gives huge jumps in q. The term k² prevents it and it's at the cost of slower convergence close to the goal.

Test goal: we took the pose of tool_link for q = [0.5, 0.3] -> position (1.148839, 0.2, 3.436532), quaternion (0, 0.389418, 0, 0.921061). In RViz the robot jumps to the random guess and then moves to the goal.

```
[test_ik-4] [INFO] [1793648415.157504746] [test_ik]: initial guess = [-0.043, 0.357]
[test_ik-4] [INFO] [1793648416.965059948] [test_ik]: converged after 88 iter
[test_ik-4] [INFO] [1793648416.965298272] [test_ik]: solution = [0.500, 0.300]
[test_ik-4] [INFO] [1793648437.399150815] [test_ik]: Rreq for frame 'tool_link' -> (1.149, 0.200, 3.
437)
[test_ik-4] [INFO] [1793648437.399294930] [test_ik]: initial guess = [1.904, 2.342]
[test_ik-4] [INFO] [1793648439.404747344] [test_ik]: converged after 98 iter
[test_ik-4] [INFO] [1793648439.404925962] [test_ik]: solution = [0.500, 6.583]
[test_ik-4] [INFO] [1793648454.178064696] [test_ik]: Rreq for frame 'tool_link' -> (1.149, 0.200, 3.
437)
[test_ik-4] [INFO] [1793648454.178204910] [test_ik]: initial guess = [-1.944, 2.046]
[test_ik-4] [INFO] [1793648456.208699999] [test_ik]: converged after 99 iter
[test_ik-4] [INFO] [1793648456.208874317] [test_ik]: solution = [0.500, 0.300]
[test_ik-4] [INFO] [1793648464.761315028] [test_ik]: Rreq for frame 'tool_link' -> (1.149, 0.200, 3.
437)
[test_ik-4] [INFO] [1793648464.761419339] [test_ik]: initial guess = [-1.425, 2.031]
[test_ik-4] [INFO] [1793648466.687384127] [test_ik]: converged after 94 iter
[test_ik-4] [INFO] [1793648466.687542644] [test_ik]: solution = [0.500, 0.300]
```

Output with λ = 0.1: every call converges in 88-99 iterations to [0.500, 0.300], once to [0.500, 6.583] (= 0.300 + 2π, potential issues).

λ	Iterations	Solutions (q1, q2)
0.1	88, 98, 99, 94	[0.500, 0.300], once [0.500, 6.583]
0.5	19, 17	[-5.783, 6.583], [0.500, 0.300]
1.0	5, 5	[0.500, 0.300]

Damping, goal from q = [0.3, 0.0, λ = 0.1:

Damping	Iterations	Solutions (q1, q2)
0	101, 118, 110	[0.300, 0.000] every time
0.1	101, 99, 101	[0.300, 0.000], [0.300, -6.283], [0.300, 6.283]

-> almost no difference.

Panda (7 DOF): we generated the URDF from pandra_arm.urdf.xacro and started test_ik with urdfPath:=/tmp/panda.urdf and λ = 1.0.

- Converged after 11 and 8 iterations
- Solutions: [13.05, -4.73, 3.01, -1.24, -12.26, -0.53, 4.04] and [-3.12, 5.08, 2.62, -11.53, 7.47, 0.67, -11.44]
- Not the q we made the goal from. Panda has 7 joints but the pose only has 6 dimensions -> redundant infinitely many solutions

Task 4: Potential issues with the algorithm

- Joint limits are ignored. Only the initial guess is inside the limits and the updates can go anywhere. We got [0.500, 6.583] for rrbot (limit [-3, 3]) and values like 13.05 and -12.26 for Panda -> not valid on a real robot. A fix could be to wrap the angles to [-π, π] and clamp them down to the limits
- Unreachable goals can never converge. Our first test goal was rounded to 3 decimals so it didn't match any reachable pose EXACTLY (rrbot has 2 joints but the goal has 3 constraints in its plane), they ended up close but the error never got below posTop -> “no convergence after 1000 iter”.
- Several solutions. Which one we get depends on the random initial guess (2π shifts for rrbot, infinitely many for the redundant Panda), so the same goal can give different joint values every time

Additional codes in the .ccp files.